

Cheat Sheet: SNA con igraph & tidygraph

Referencia rápida para examen — Temas 7, 8 y 9

Table of contents

1. Construcción de Grafos (igraph)	2
Desde distintos formatos	2
Info básica del grafo	3
2. Atributos de Nodos y Aristas	3
3. Visualización con igraph (plot)	4
Estructura básica	4
Layouts disponibles	4
Grupos visuales (<code>mark.groups</code>)	5
Coloreado condicional de aristas (por facción)	5
4. Paradigma Tidygraph	5
Conversión y activación	5
Verbos dplyr en red	6
Pronombre <code>.N()</code> — cruzar nodos desde aristas	6
Morphing — vistas temporales	6
Modificaciones estructurales	7
5. Visualización con ggraph	7
Arquitectura de capas	7
Geoms de aristas más usados	7
Geoms de nodos	8
NSE — calcular métricas directamente en <code>aes()</code>	8
Envolventes de comunidades con <code>ggforce</code>	8
Gradiente de dirección en grafos dirigidos	9
Facetas	9

6. Métricas de Centralidad	9
Con <code>igraph</code> (funciones base)	9
Con <code>tidygraph</code> (dentro de <code>mutate</code>)	10
Cuándo usar cada métrica	10
7. Análisis Estructural Global	10
8. Detección de Comunidades	11
Algoritmos disponibles	11
Cuándo usar cada algoritmo	11
Visualización rápida de comunidades	12
9. Redes Egocéntricas	12
10. Exportación	12
Referencia rápida de operadores <code>igraph</code>	13

1. Construcción de Grafos (`igraph`)

Desde distintos formatos

```
library(igraph)

# 1. Lista de aristas (pares de nodos)
make_graph(edges = c(1,2, 2,3, 3,1))      # por índices
make_graph(~ A-B-C-A)                     # por fórmula
make_graph("Zachary")                     # grafo famoso

# 2. Construcción secuencial (vacío + añadir)
g <- make_empty_graph(n = 0, directed = TRUE) +
  vertices(c("A","B","C")) +
  edges(c("A","C", "B","C"))
g <- g - V(g)["A"]                         # eliminar nodo

# 3. Desde data frame (más habitual en datos reales)
df <- data.frame(from = c("A","B"), to = c("B","C"), weight = c(1,2))
g <- graph_from_data_frame(df, directed = FALSE)

# 4. Desde matriz de adyacencia
A <- matrix(c(0,1,1, 1,0,0, 1,0,0), nrow=3, byrow=TRUE)
```

```

g <- graph.adjacency(A, mode = "undirected") # → grafo
A <- as_adjacency_matrix(g, sparse = FALSE) # grafo → matriz

# 5. Grafos aleatorios / sintéticos
g_er <- sample_gnp(10, 0.3, directed = FALSE) # Erdős-Rényi
g_ba <- sample_pa(100, power = 1.2, m = 2) # Barabási-Albert
make_tree(20, children = 3)
make_full_graph(6) # clique
make_star(10)
make_lattice(dimvector = c(5,5))

```

Info básica del grafo

```

vcount(g) # n° nodos
ecount(g) # n° aristas
is.directed(g) # TRUE/FALSE
is_simple(g) # sin bucles ni aristas múltiples

```

2. Atributos de Nodos y Aristas

```

# Leer / asignar atributos
V(g)$name <- paste0("N", 1:vcount(g)) # nombre
V(g)$color <- "lightblue"
V(g)$size <- 15
V(g)$shape <- "circle" # "circle", "rectangle"

E(g)$weight <- sample(1:5, ecount(g), replace = TRUE)
E(g)$width <- E(g)$weight * 0.5
E(g)$color <- "gray80"
E(g)$curved <- 0.2

# Asignación condicional
V(g)[c(1,34)]$color <- "orange"
isVowel <- V(g)$name %in% c("A","E","I","O","U")
V(g)[isVowel]$shape <- "square"

```

```
# Selección de aristas
E(g)[v1 %--% v1]$width <- 4      # bidireccional (ambas dir.)
E(g)[v1 %->% v2]$color <- "red"  # solo A→B
E(g)[v1 %-<% v2]                  # solo B→A
which_mutual(g)                   # vector lógico aristas mutuas
```

3. Visualización con igraph (plot)

Estructura básica

```
plot(g,
  layout          = layout_with_fr(g),  # algoritmo de disposición
  vertex.size     = 15,
  vertex.color    = V(g)$color,
  vertex.shape    = "circle",
  vertex.label    = V(g)$name,
  vertex.label.cex = 0.8,
  vertex.label.color = "black",
  vertex.label.dist = 1.2,              # aleja la etiqueta del centro
  vertex.frame.color = "white",
  edge.color      = E(g)$color,
  edge.width      = E(g)$width,
  edge.curved     = 0.2,
  edge.arrow.size = 0.5,                # solo grafos dirigidos
  main = "Título")
```

Layouts disponibles

Función	Cuándo usarla
layout_with_fr	Estándar — revela clusters (Fruchterman-Reingold)
layout_with_kk	Distancias proporcionales a caminos cortos (Kamada-Kawai)
layout_in_circle	Redes densas o cíclicas
layout_on_grid	Inspección individual, sin importar topología

Función	Cuándo usarla
---------	---------------

Grupos visuales (mark.groups)

```
groupList <- list(g1 = which(isVowel), g2 = c(2,3,4))
plot(g, layout = layout_with_fr,
     mark.groups = groupList,
     mark.col     = c(rgb(0,0.3,1,0.5), rgb(0.8,0.4,0.1,0.5)),
     mark.border  = NA)
```

Coloreado condicional de aristas (por facción)

```
edge_nodes <- as_edgelist(g, names = FALSE)
edge_colors <- sapply(1:ecount(g), function(i) {
  if (V(g)$Faction[edge_nodes[i,1]] == V(g)$Faction[edge_nodes[i,2]])
    return("skyblue")
  else return("gray80")
})
plot(g, edge.color = edge_colors)
```

4. Paradigma Tidygraph

Conversión y activación

```
library(tidygraph)

karate_tidy <- as_tbl_graph(karate)      # igraph → tbl_graph
.active(karate_tidy)                    # ver tabla activa ("nodes"/"edges")

# Activar tabla de nodos o aristas
karate_tidy %>% activate(nodes) %>% ...
karate_tidy %>% activate(edges) %>% ...
```

Verbos dplyr en red

```
grafo <- karate_tidy %>%
  activate(nodes) %>%
  mutate(
    grado      = centrality_degree(),
    comunidad = as.factor(group_infomap()),
    es_lider   = grado > 15
  ) %>%
  filter(!node_is_isolated()) %>%      # elimina nodos sin conexión
  activate(edges) %>%
  mutate(puente = centrality_edge_betweenness()) %>%
  filter(puente > 2)
```

Pronombre .N() — cruzar nodos desde aristas

```
karate_tidy %>%
  activate(edges) %>%
  mutate(
    fac_origen = .N()$Faction[from], # atributo del nodo origen
    fac_destino = .N()$Faction[to],   # atributo del nodo destino
    es_cruce   = fac_origen != fac_destino
  )
```

Morphing — vistas temporales

```
# Patrón: morph → operar → unmorph
grafo <- karate_tidy %>%
  morph(to_shortest_path, from = 1, to = 34) %>%
  activate(nodes) %>% mutate(en_ruta = TRUE) %>%
  activate(edges) %>% mutate(en_ruta = TRUE) %>%
  unmorph()

# crystallise() → lista de tbl_graph independientes
lista_comps <- karate_tidy %>% morph(to_components) %>% crystallise()
```

Morpher	Resultado
<code>to_shortest_path(from, to)</code>	Camino mínimo entre dos nodos
<code>to_local_neighborhood(node, order)</code>	Ego-red de orden n
<code>to_contracted(atributo)</code>	Colapsa grupos en súper-nodos
<code>to_components()</code>	Subgrafos inconexos separados
<code>to_minimum_spanning_tree()</code>	Esqueleto mínimo de conectividad

Modificaciones estructurales

```
bind_nodes(data.frame(name = "Nuevo", Faction = 1)) # añadir nodo
bind_edges(data.frame(from = 35, to = 1))           # añadir arista
graph_join(red_extra, by = "name")                  # fusionar dos redes
activate(edges) %>% slice(1) %>% reroute(from=1, to=2) # redirigir arista
```

5. Visualización con ggraph

Arquitectura de capas

```
library(ggraph)

ggraph(g_tidy, layout = "fr") + # "fr", "kk", "stress", "circle", "tree"
  geom_edge_link(alpha = 0.3) +
  geom_node_point(aes(size = centrality_degree(),
                      color = factor(group_louvain())) +
  scale_color_viridis_d(option = "plasma") +
  theme_graph() +
  labs(title = "Mi red")
```

Geoms de aristas más usados

Geom	Cuándo usarlo
<code>geom_edge_link()</code>	Caso general — línea recta

Geom	Cuándo usarlo
<code>geom_edge_fan()</code>	Múltiples aristas entre mismos nodos (multígrafo)
<code>geom_edge_parallel()</code>	Red multicapa — desplaza aristas por tipo
<code>geom_edge_diagonal()</code>	Estructuras jerárquicas / árboles
<code>geom_edge_arc()</code>	Layout circular — aristas curvadas

Geoms de nodos

```
geom_node_point(aes(size = centrality_degree(), color = comunidad))
geom_node_text(aes(label = name), check_overlap = TRUE, vjust = 1.5)
geom_node_label(aes(label = name))
```

NSE — calcular métricas directamente en aes()

```
# Sin necesidad de mutate previo
geom_node_point(aes(
  size = centrality_degree(),
  color = as.factor(group_infomap())
))
geom_edge_link(aes(alpha = centrality_edge_betweenness()))
```

Envoltentes de comunidades con ggforce

```
library(ggforce)

g_tidy <- as_tbl_graph(karate) %>%
  mutate(comunidad = as.factor(group_louvain()))

ggraph(g_tidy, layout = "nicely") +
  geom_mark_hull(aes(x = x, y = y, fill = comunidad, group = comunidad),
    alpha = 0.15, color = NA, concavity = 10, expand = unit(2, "mm")) +
  geom_edge_link(alpha = 0.2, color = "gray70") +
  geom_node_point(aes(color = comunidad), size = 4) +
  scale_fill_brewer(palette = "Set1") +
```



```
scale_color_brewer(palette = "Set1") +
theme_graph() + theme(legend.position = "none")
```

Gradiente de dirección en grafos dirigidos

```
geom_edge_fan(aes(alpha = after_stat(index)), color = "darkred",
              show.legend = FALSE)
# El alpha aumenta desde origen → destino, indicando dirección
```

Facetas

```
g_tidy %>% activate(nodes) %>%
  mutate(Sede = sample(c("Norte", "Sur"), n(), replace = TRUE)) %>%
  ggraph("stress") +
  geom_edge_link(alpha = 0.4) +
  geom_node_point(color = "orange", size = 4) +
  facet_nodes(~ Sede) + theme_graph()
```

6. Métricas de Centralidad

Con igraph (funciones base)

<code>degree(g)</code>	# grado total
<code>degree(g, mode = "in")</code>	# grado de entrada (dirigido)
<code>degree(g, mode = "out")</code>	# grado de salida (dirigido)
<code>betweenness(g)</code>	# intermediación
<code>closeness(g)</code>	# cercanía (¡cuidado en dirigidos!)
<code>harmonic_centrality(g)</code>	# cercanía armónica (robusta)
<code>eigen_centrality(g)\$vector</code>	# eigenvector (influencia por vecinos)
<code>hub_score(g)\$vector</code>	# HITS: hubs
<code>authority_score(g)\$vector</code>	# HITS: authorities

Closeness en dirigidos: usar `harmonic_centrality()` — penaliza correctamente nodos aislados (evita el “efecto isla”).

Con tidygraph (dentro de mutate)

```
mutate(  
  degree      = centrality_degree(),  
  betweenness = centrality_betweenness(),  
  closeness   = centrality_closeness(),  
  pagerank    = centrality_pagerank(),  
  eigen       = centrality_eigen(),  
  edge_btw    = centrality_edge_betweenness() # sobre aristas  
)
```

Cuándo usar cada métrica

Métrica	Objetivo	Ejemplo de uso
Degree	Popularidad, actividad local	Influencers, nodos con alta carga
Betweenness	Control de flujo, SPOF	Puntos únicos de fallo, gatekeepers
Closeness	Rapidez de difusión	Dónde poner caché, vacunación
PageRank / Eigenvector	Prestigio por calidad de vecinos	Líderes de opinión
HITS Hubs	Nodos que apuntan a buenas autoridades	Medios/agregadores
HITS Authorities	Nodos muy referenciados por buenos hubs	Productores de contenido

7. Análisis Estructural Global

```
diameter(g) # camino más corto más largo  
shortest_paths(g, from = "1", to = "10")$vpath # ruta mínima  
edge_betweenness(g) # betweenness de aristas  
edge_density(g) # densidad (aristas / posibles)  
  
# Clustering (transitividad)
```

```

transitivity(g, type = "global")      # C global
transitivity(g, type = "local")      # C por nodo

# nodos con C 0 son brokers (agujeros estructurales)
brokers <- which(transitivity(g, type = "local") < 0.1)

```

Interpretación del clustering local:

- $C = 1 \rightarrow$ clique perfecta (todos mis vecinos se conocen entre sí)
- $C = 0 \rightarrow$ agujero estructural (el nodo es el único puente entre grupos)

8. Detección de Comunidades

Algoritmos disponibles

```

# igraph base
cluster_edge_betweenness(g)  # elimina puentes progresivamente
cluster_walktrap(g)         # random walks (caminante atrapado)
cluster_fast_greedy(g)      # bottom-up, optimiza modularidad
cluster_leading_eigen(g)    # top-down, vectores propios
cluster_label_prop(g)       # propagación de etiquetas (muy rápido)
cluster_louvain(g)          # Louvain - escalable, gran escala

# tidygraph (dentro de mutate)
mutate(comunidad = group_louvain())
mutate(comunidad = group_infomap())
mutate(comunidad = group_edge_betweenness())
mutate(comunidad = group_walktrap())
mutate(comunidad = group_label_prop())

```

Cuándo usar cada algoritmo

Algoritmo	Escala	Criterio de elección
Louvain	Grande	Velocidad + calidad general

Algoritmo	Escala	Criterio de elección
Infomap	Mediana	Flujos de información (teoría de la información)
Edge Betweenness	Pequeña	Interpretabilidad, puentes explícitos
Label Propagation	Muy grande	Máxima velocidad, grafos masivos

Visualización rápida de comunidades

```
comm <- cluster_louvain(g)
plot(comm, g, main = "Comunidades") # igraph base con colores automáticos
```

9. Redes Egocéntricas

```
# Ego-red de orden 1 (yo + mis contactos directos)
ego(g, order = 1, nodes = 1)

# Con tidygraph/morphing
grafo_ego <- karate_tidy %>%
  morph(to_local_neighborhood, node = 1, order = 2) %>%
  mutate(es_vecino = TRUE) %>%
  unmorph()
```

10. Exportación

```
write_graph(g, "red.graphml", format = "graphml") # formato XML estándar
```

Referencia rápida de operadores igraph

Operador	Función
<code>V(g)</code>	Secuencia de vértices
<code>E(g)</code>	Secuencia de aristas
<code>V(g)[cond]</code>	Filtrar vértices por condición
<code>E(g)[A %--% B]</code>	Aristas entre A y B (sin dir.)
<code>E(g)[A %->% B]</code>	Aristas de A hacia B
<code>E(g)[A %<-% B]</code>	Aristas de B hacia A
<code>as_edgelist(g)</code>	Matriz (from, to) de aristas
<code>which_mutual(g)</code>	Vector lógico de aristas mutuas
<code>node_is_isolated()</code>	Nodos sin ninguna conexión (tidygraph)